SISTEMA DE GESTIÓN DE BIBLIOTECA

INFORME TÉCNICO - ACTIVIDAD DE CONSTRUCCIÓN APLICADA (ACA)

Asignatura: Programación Avanzada **Ficha/ Grupo:** 54463 **Programa:** Ingeniería de Sistemas **Estudiante:** Jorge Humberto Muñoz Ramos **Docente:** Verónica Castro Munar **Fecha:** noviembre de 2026 **Tecnologías:** C#, .NET Windows Forms, SQL Server, ADO.NET, Git/GitHub

ENLACE REPOSITORIO GITHUB

https://github.com/jhmunoz94/ACA-PROGRAMACI-N-AVANZADA.git

TABLA DE CONTENIDO

1. Portada y Presentación
2. Introducción
3. Objetivos (General y Específicos)
4. Planteamiento del Problema
5. Análisis de Requerimientos (Funcionales y No Funcionales)
6. Casos de Uso Principales
7. Arquitectura del Sistema (Patrón en 3 Capas)
8. Diagrama de Clases y Entidades
9. Modelo Entidad-Relación (DER) y Diccionario de Datos
10. Conexión y Persistencia en SQL Server
11. Pruebas y Evidencias de Funcionamiento
12. Conclusiones y Recomendaciones
13. Referencias Bibliográficas (Normas APA)

---

1. INTRODUCCIÓN

El presente proyecto documenta el diseño, construcción e implementación de una solución informática integral denominada **Sistema de Gestión de Biblioteca**, desarrollada como parte del proceso formativo de la asignatura Programación Avanzada. La aplicación responde a la necesidad de controlar de forma confiable, centralizada y automatizada el flujo diario de material bibliográfico, el registro de usuarios y el ciclo operativo de préstamos y devoluciones en un entorno institucional.

La solución se concibió bajo las mejores prácticas de la ingeniería de software, aplicando el paradigma de la Programación Orientada a Objetos (POO), desacoplamiento modular en tres capas lógicas (Presentación, Negocio y Datos) y persistencia relacional estricta en Microsoft SQL Server.

---

2. OBJETIVOS

2.1. Objetivo General

Desarrollar una aplicación de escritorio modular para la gestión integral de una biblioteca utilizando C#, Windows Forms y Microsoft SQL Server, que permita administrar libros, autores, editoriales y usuarios, controlar préstamos y devoluciones con validación física de stock en tiempo real, y emitir reportes consolidados.

2.2. Objetivos Específicos

- Modelar e implementar una base de datos relacional normalizada en SQL Server garantizando integridad referencial mediante claves primarias y foráneas.
- Implementar operaciones CRUD (Crear, Leer, Actualizar y Eliminar) completas y parametrizadas para evitar riesgos de inyección SQL.
- Desarrollar una arquitectura de software desacoplada en 3 capas, separando la interfaz visual de las reglas de negocio y de los mecanismos de acceso a datos.
- Validar en la capa de negocio las reglas de dominio institucional: disponibilidad de stock, unicidad de documentos/ISBN y obligatoriedad de campos.
- Integrar un módulo de reportería y consultas en tiempo real filtrado por colecciones genéricas con LINQ.
- Versionar el ciclo de vida del código mediante Git y GitHub.

---

3. PLANTEAMIENTO DEL PROBLEMA

El control empírico o manual de inventarios bibliográficos genera inconsistencias operativas críticas: pérdida física de ejemplares, asignación de préstamos sobre material agotado, duplicidad de registros y ausencia de métricas sobre préstamos vencidos o activos. Para solventar estas deficiencias, se requiere una herramienta de escritorio segura, intuitiva y conectada a un motor relacional que impida estados inconsistentes y garantice trazabilidad inmediata.

---

4. ANÁLISIS DE REQUERIMIENTOS

4.1. Requerimientos Funcionales (RF)

- **RF01 - Gestión de Libros:** Registrar, listar, modificar y eliminar material bibliográfico vinculado a un autor y una editorial con control de stock.
- **RF02 - Gestión de Autores:** Administrar nombres, apellidos y país de origen de autores.
- **RF03 - Gestión de Editoriales:** Controlar firmas editoriales asociadas al catálogo.
- **RF04 - Gestión de Usuarios:** Registrar y actualizar datos personales, documento de identidad y medios de contacto.
- **RF05 - Control de Préstamos:** Registrar salidas de libros asociando usuario y material bibliográfico, comprobando que $Stock > 0$.
- **RF06 - Control de Devoluciones:** Registrar la recepción de ejemplares prestados, actualizando el estado a "Devuelto" y sumando una unidad al stock.
- **RF07 - Reportes y Consultas:** Visualizar reportes en tiempo real clasificados por préstamos activos, devueltos, catálogo general y padrón de usuarios.

- **RF08 - Navegación Modular:** Disponer de una interfaz contenedora (`FormMenu`) con panel lateral de acceso responsivo.

4.2. Requerimientos No Funcionales (RNF)

- **Arquitectura:** División estricta en capas (Presentación $\rightarrow$ Negocio $\rightarrow$ Acceso a Datos $\rightarrow$ Entidades).
- **Seguridad:** Consultas transaccionales parametrizadas (`SqlParameter`) para neutralizar inyección SQL.
- **Robustez:** Control exhaustivo de excepciones (`try-catch-finally`) con mensajes amigables al usuario.
- **Rendimiento:** Apertura y cierre explícito de conexiones SQL mediante sentencias `using`.

---

5. CASOS DE USO PRINCIPALES

- **CU01 - Préstamo de Libro:**

Actor: Bibliotecario / Administrador. *Flujo:* El usuario selecciona el lector y el libro desde listas desplegables. La capa de negocio verifica el stock. Si $Stock \ge 1$, se inserta el préstamo en estado 'Prestado', se reduce el stock en la tabla `Libros` en 1 unidad y se refresca la grilla. Si $Stock = 0$, se bloquea la transacción informando la falta de ejemplares.

- **CU02 - Devolución de Libro:**

Actor: Bibliotecario. *Flujo:* Se selecciona el préstamo desde el DataGridView. Se valida que el estado no sea previamente 'Devuelto'. Se sella la fecha de devolución actual, se conmuta el estado a 'Devuelto' y se reintegra (+1) el stock del libro involucrado.

- **CU03 - Alta de Libro:**

Actor: Bibliotecario. *Flujo:* Se digita el ISBN, título, año y stock inicial, vinculándolos obligatoriamente a un Autor y Editorial existentes en el sistema.

---

6. ARQUITECTURA DEL SISTEMA (PATRÓN EN 3 CAPAS)

La aplicación implementa el patrón de capas desacopladas en C#:

14. **Capa de Presentación (`Biblioteca.Presentacion`):**
    - Formularios Windows Forms (`FormMenu`, `FormLibros`, `FormUsuarios`, `FormAutores`, `FormEditoriales`, `FormPrestamos`, `FormReportes`).
    - No interactúa directamente con la base de datos; delega los eventos a la capa de negocio y renderiza grillas responsivas y controles centrados.
15. **Capa de Lógica de Negocio (`Biblioteca.Negocio`):**
    - Clases controladoras (`LibroNegocio`, `UsuarioNegocio`, `AutorNegocio`, `EditorialNegocio`, `PrestamoNegocio`).
    - Valida restricciones lógicas institucionales: campos no vacíos, formato numérico de año, existencias para préstamos y coherencia de estados.
16. **Capa de Acceso a Datos (`Biblioteca.Datos`):**

- Encapsula la infraestructura de base de datos a través de ADO.NET (`Conexion.cs`, `LibroDatos`, etc.).
- Ejecuta transacciones SQL, sentencias DML y lectura eficiente por medio de `SqlDataReader` y `SqlDataAdapter`.

17. **Capa Transversal de Entidades (`Biblioteca.Entidades`):**
    - Modelos POCO (`Libro.cs`, `Usuario.cs`, `Autor.cs`, `Editorial.cs`, `Prestamo.cs`) que transportan las estructuras de datos entre capas con constructores y propiedades encapsuladas `{ get; set; }`.

---

7. DIAGRAMA DE CLASES Y MODELO ENTIDAD-RELACIÓN (DER)

7.1. Diccionario de Datos Relacional

Tabla `Autores`

Campo	Tipo de Dato	Nulidad	Restricción / Rol
`IdAutor`	INT	NOT NULL	Clave Primaria (PK), IDENTITY(1,1)
`Nombre`	VARCHAR(100)	NOT NULL	Nombre de pila del autor
`Apellido`	VARCHAR(100)	NOT NULL	Apellido del autor
`Nacionalidad`	VARCHAR(50)	NULL	País de origen

Tabla `Editoriales`

Campo	Tipo de Dato	Nulidad	Restricción / Rol
`IdEditorial`	INT	NOT NULL	Clave Primaria (PK), IDENTITY(1,1)
`Nombre`	VARCHAR(100)	NOT NULL	Clave Única (UNIQUE)
`Pais`	VARCHAR(50)	NULL	Ubicación geográfica

Tabla `Usuarios`

Campo	Tipo de Dato	Nulidad	Restricción / Rol
`IdUsuario`	INT	NOT NULL	Clave Primaria (PK), IDENTITY(1,1)
`Documento`	VARCHAR(20)	NOT NULL	Clave Única (UNIQUE), Documento de Identidad
`Nombre`	VARCHAR(100)	NOT NULL	Nombre del usuario
`Apellido`	VARCHAR(100)	NOT NULL	Apellido del usuario
`Telefono`	VARCHAR(20)	NULL	Teléfono celular de contacto
`Correo`	VARCHAR(100)	NULL	Correo electrónico

Tabla `Libros`

Campo	Tipo de Dato	Nulidad	Restricción / Rol
IdLibro	INT	NOT NULL	Clave Primaria (PK), IDENTITY(1,1)
CodigoISBN	VARCHAR(20)	NOT NULL	Clave Única (UNIQUE), Código normalizado
Titulo	VARCHAR(200)	NOT NULL	Nombre del libro
AnioPublicacion	INT	NOT NULL	Año de publicación
Stock	INT	NOT NULL	Cantidad de existencias físicas disponibles (≥ 0)
IdAutor	INT	NOT NULL	Clave Foránea (FK) $\rightarrow$ Autores(IdAutor)
IdEditorial	INT	NOT NULL	Clave Foránea (FK) $\rightarrow$ Editoriales(IdEditorial)

Tabla Prestamos

Campo	Tipo de Dato	Nulidad	Restricción / Rol
IdPrestamo	INT	NOT NULL	Clave Primaria (PK), IDENTITY(1,1)
IdUsuario	INT	NOT NULL	Clave Foránea (FK) $\rightarrow$ Usuarios(IdUsuario)
IdLibro	INT	NOT NULL	Clave Foránea (FK) $\rightarrow$ Libros(IdLibro)
FechaPrestamo	DATETIME	NOT NULL	Marca de tiempo de salida (DEFAULT GETDATE())
FechaDevolucion	DATETIME	NULL	Marca de tiempo de recepción efectiva
Estado	VARCHAR(20)	NOT NULL	'Prestado' o 'Devuelto'

---

8. CONEXIÓN CON SQL SERVER

La persistencia de datos opera mediante un proveedor de datos System.Data.SqlClient. La cadena de conexión se centraliza en una clase utilitaria desacoplada:

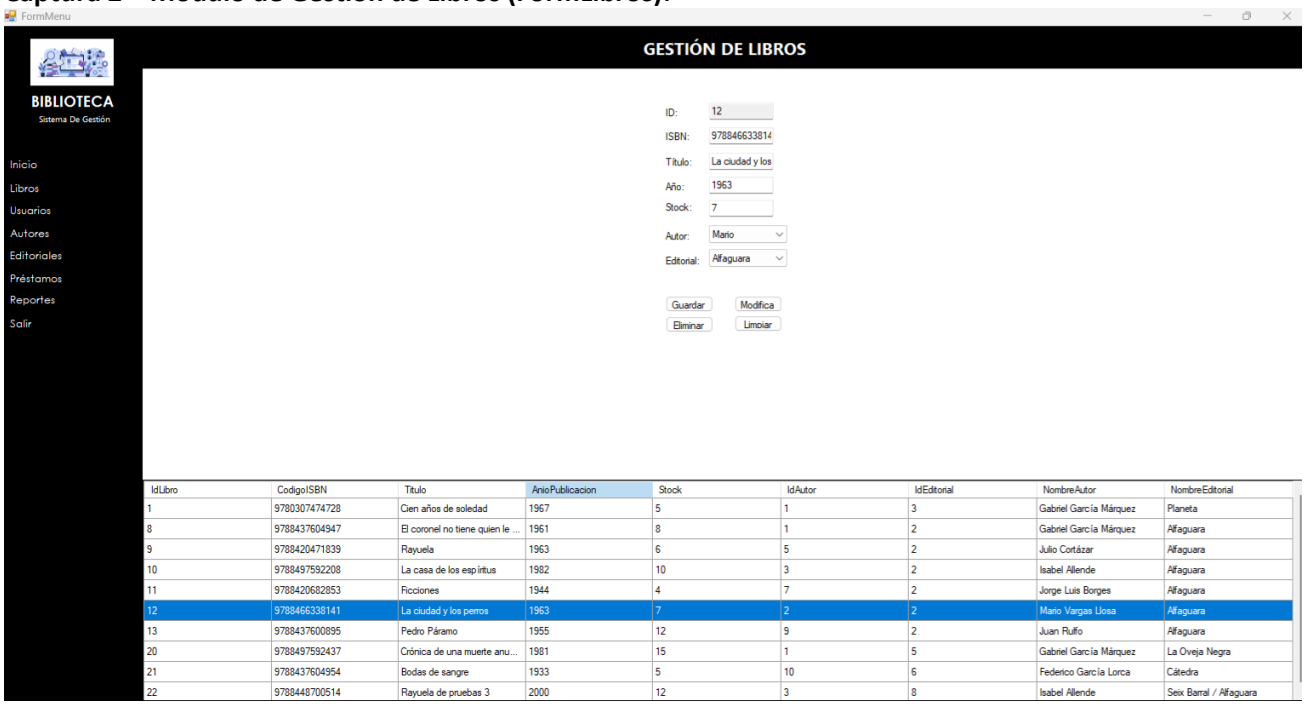
```
Data Source=localhost;Initial Catalog=BibliotecaDB;Integrated Security=True;
```

9. Pruebas y Evidencias de Funcionamiento

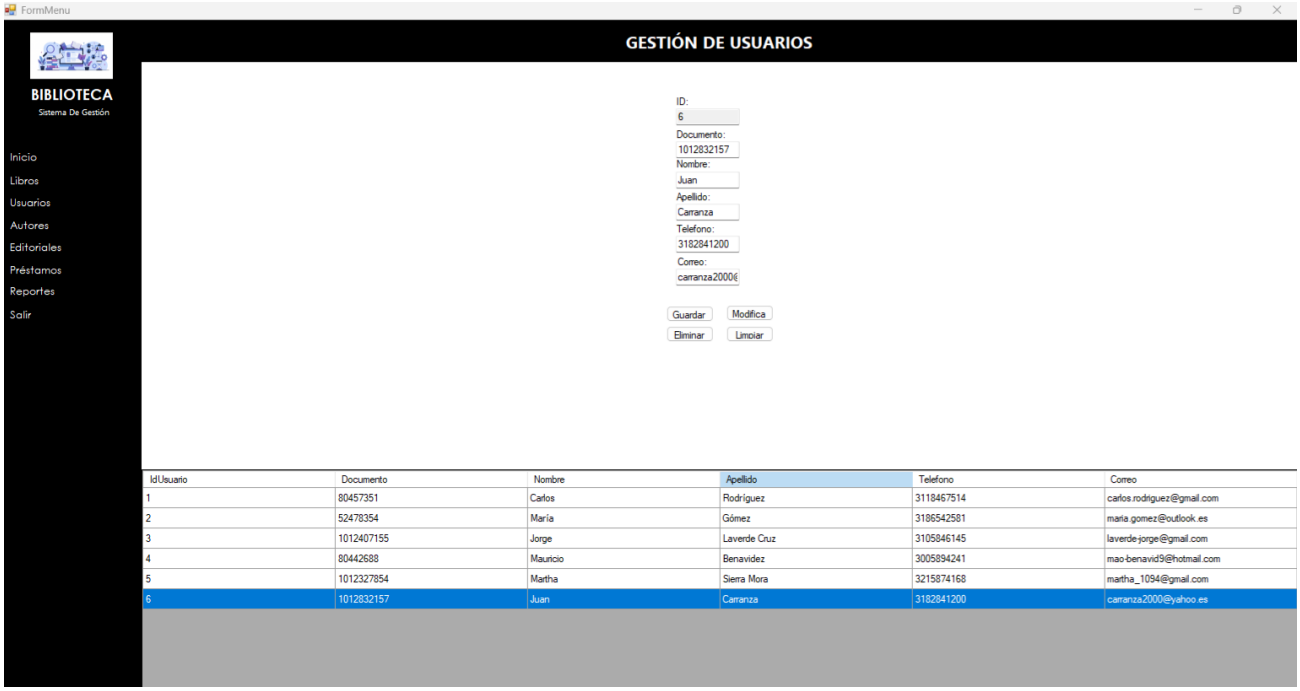
Captura 1 – Pantalla de Inicio / Menú Principal (FormMenu):



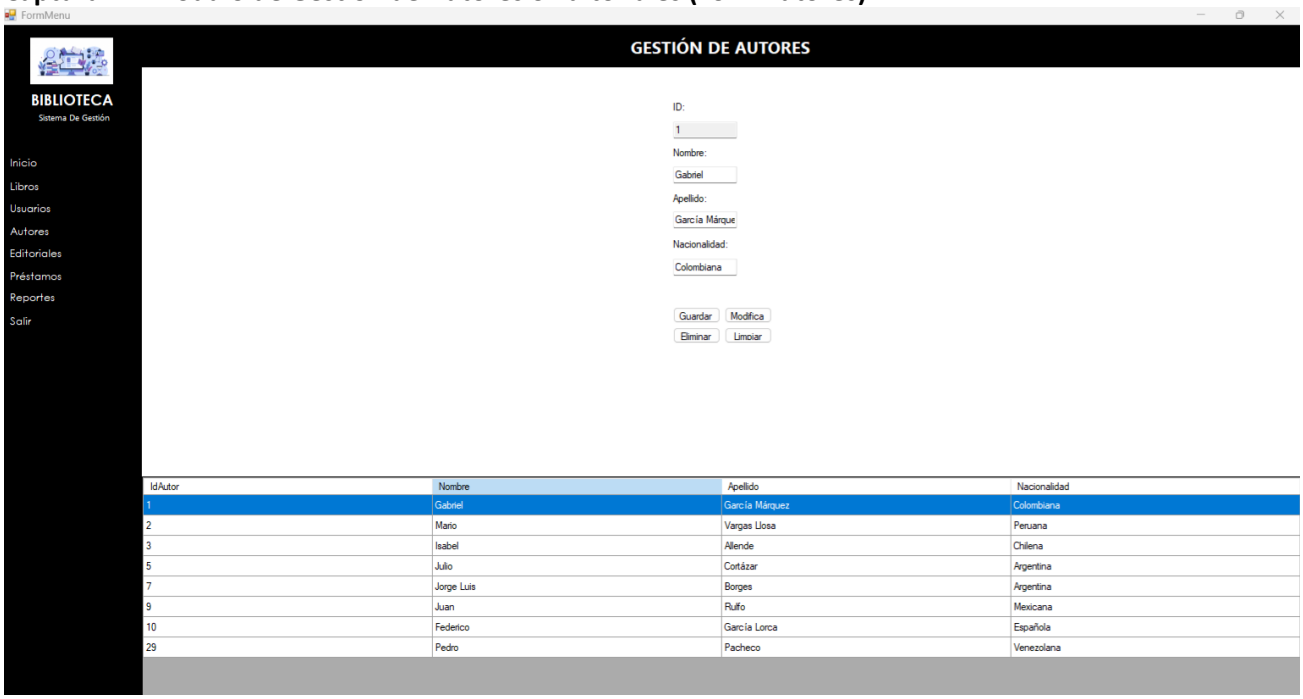
Captura 2 – Módulo de Gestión de Libros (FormLibros):

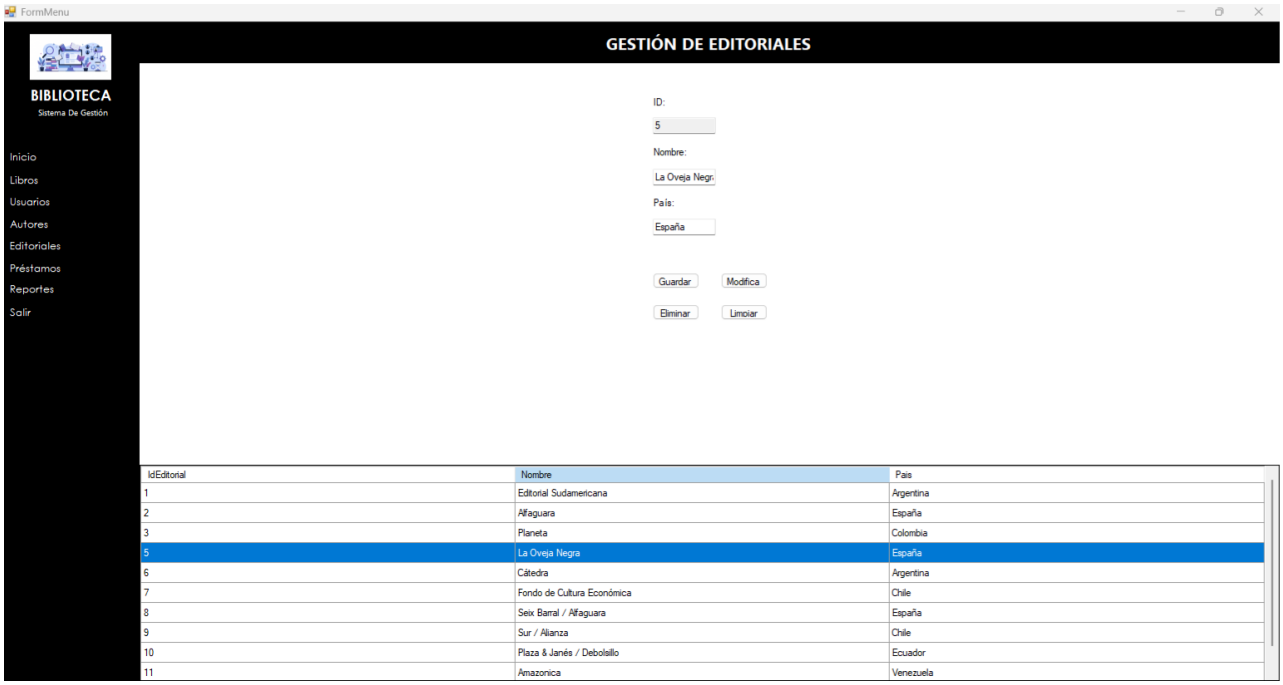


Captura 3 – Módulo de Gestión de Usuarios (FormUsuarios):

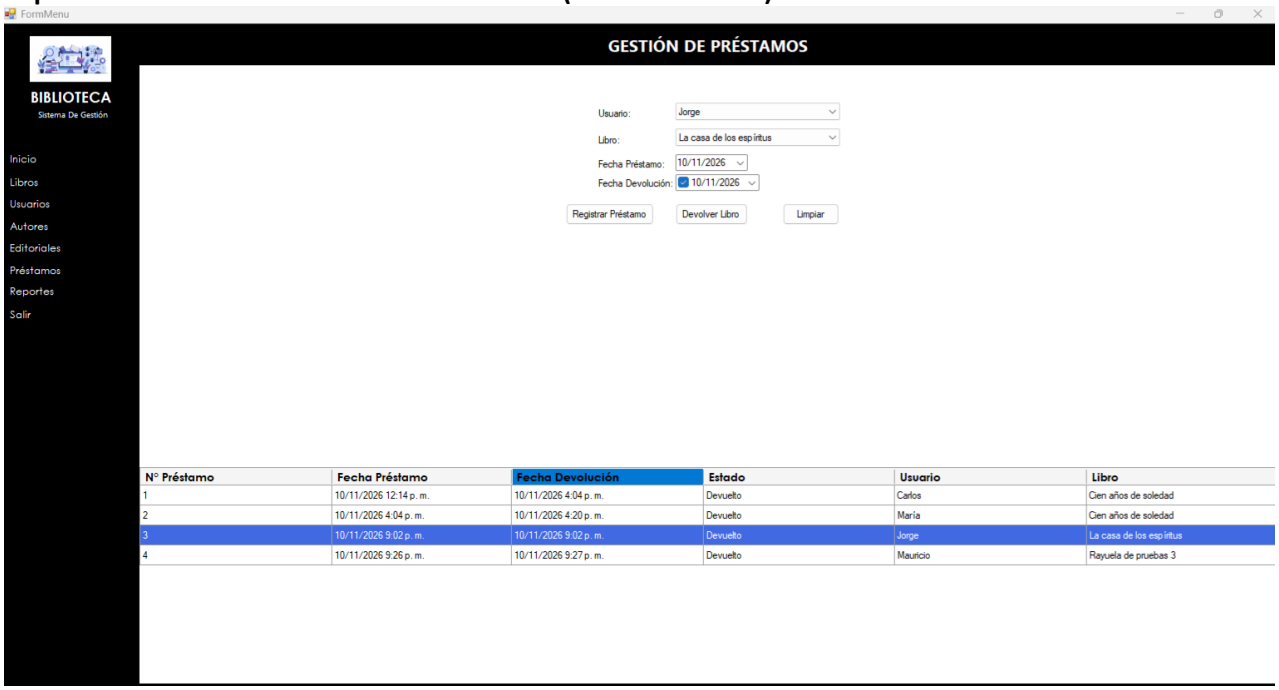


Captura 4 – Módulo de Gestión de Autores o Editoriales (FormAutores):

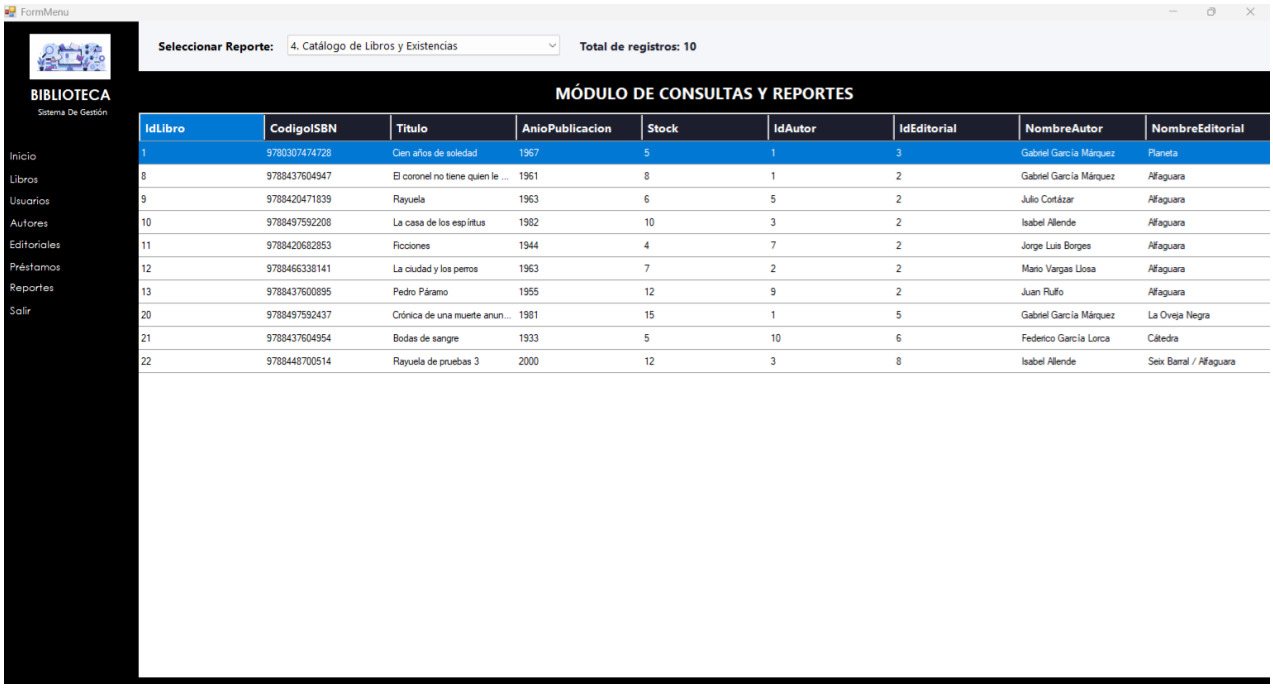




Captura 5 – Módulo de Gestión de Préstamos (FormPrestamos):



Captura 6 – Módulo de Consultas y Reportes (FormReportes):



10. CONCLUSIONES Y RECOMENDACIONES

- La arquitectura multicapa demostró ser la metodología óptima para desacoplar interfaces gráficas de la lógica de persistencia, facilitando el mantenimiento correctivo y preventivo sin afectar al usuario final.

- La normalización del esquema relacional en SQL Server protegió la integridad referencial, impidiendo la existencia de préstamos huérfanos o catálogos desvinculados de autores.

- La validación en capa de negocio evitó el bloqueo de concurrencia e impidió la degradación del stock bibliográfico.

- Se recomienda para versiones futuras implementar roles de usuario (administrador, bibliotecario, estudiante) con autenticación encriptada mediante algoritmos hash (BCrypt o SHA-256).

11. REFERENCIAS BIBLIOGRÁFICAS (NORMAS APA)

- Freeman, A. (2020). Pro C# 9 with .NET 5: Foundational Principles and Practices in Programming. Apress.

- Microsoft Learn. (2026). Arquitectura de aplicaciones para Windows Forms y conexión con ADO.NET en C#. Microsoft Documentation.

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.